

CodeAlpha Python Programming Internship

Task 2 Solution: Stock Portfolio Tracker Documentation & Source Code

1. Project Objective & Scope

The objective of this task is to design a terminal-driven application to manage and analyze a basic financial equity portfolio. As detailed in the provided instructions, the implementation covers:

- Accepting stock ticker labels and ownership quantities via console inputs.
- A hardcoded stock pricing reference dictionary representing live market valuations.
- Calculation and tabular aggregation of user equity valuation metrics.
- File persistent capability saving summaries into a localized .txt log file.

Key Concepts Demonstrated: Dictionary mapping, structured console input streams, fundamental arithmetic computation, and native Python file handling streams.

2. Python Source Code

```
def run_portfolio_tracker():
    # Hardcoded stock price lookup dictionary as per instructions
    STOCK_PRICES = {
        "AAPL": 180.00,
        "TSLA": 250.00,
        "GOOGL": 150.00,
        "MSFT": 420.00,
        "AMZN": 175.00
    }

    portfolio = {}
    print("=== Welcome to CodeAlpha Stock Portfolio Tracker ===")
    print("Available tickers and current market prices:")
    for ticker, price in STOCK_PRICES.items():
        print(f" - {ticker}: ${price:.2f}")
    print()

    while True:
        ticker = input("Enter stock ticker to add (or type 'done' to finish):")
        ticker = ticker.strip().upper()
```

```

    if ticker == "DONE":
        break

    if ticker not in STOCK_PRICES:
        print(f"⚠️ Ticker '{ticker}' is not supported in this simplified
version. Try again.")
        continue

    try:
        quantity = int(input(f"Enter quantity for {ticker}: "))
        if quantity <= 0:
            print("⚠️ Quantity must be greater than zero.")
            continue
    except ValueError:
        print("⚠️ Invalid input. Quantity must be an integer.")
        continue

    # Add or update portfolio holding
    portfolio[ticker] = portfolio.get(ticker, 0) + quantity
    print(f"✅ Added {quantity} shares of {ticker} to your portfolio.
")

    # Calculate investment portfolio details
    print("
=== Your Portfolio Investment Summary ===")
    total_portfolio_value = 0.0
    report_lines = []

    report_lines.append("Ticker      Quantity      Market Price      Total Value")
    report_lines.append("-----")

    for ticker, qty in portfolio.items():
        price = STOCK_PRICES[ticker]
        position_value = qty * price
        total_portfolio_value += position_value
        line = f"{ticker:<9}{qty:<12}${price:<15.2f}${position_value:.2f}"
        report_lines.append(line)

    report_lines.append("-----")
    report_lines.append(f"Total Estimated Portfolio Value: $
{total_portfolio_value:.2f}")

    # Display to standard output
    for line in report_lines:
        print(line)

```

```

# Save report summary to a .txt file
file_name = "portfolio_summary.txt"
try:
    with open(file_name, "w") as f:
        f.write("
.join(report_lines) + "
")
        print(f"
📁 Portfolio successfully saved to localized file: '{file_name}')
```

```

    except IOError:
        print("❌ Error: Unable to save your summary to file system.")

if __name__ == "__main__":
    run_portfolio_tracker()
```

3. Architecture Breakdown

- **Asset Registry Lookup:** Leveraging key-value pair indices inside hash-maps (Python dictionaries) yields instant $O(1)$ asset price tracking.
- **Persistent Streaming:** Implements defensive execution contexts using `with open()` keywords to guarantee underlying disk streams close safely regardless of run-time hardware errors.